

A Digital-Twin Simulation and AI Decision-Support System for E-Bike Battery-Swap Locker Networks

Louis Battle^{*,a,1}^aE-Bike Battery-Swap Network Simulation Project

Preliminary beta report, compiled June 10, 2026

Abstract

Urban e-bike delivery is creating fast-growing demand for battery-swap locker infrastructure, yet deciding *where*, *how many*, and *what size* lockers to deploy is largely intuition-driven and expensive to test in the field. We present a digital-twin, agent-based simulation of a battery-swap network built on the real Amsterdam street network, together with an end-to-end machine-learning pipeline that learns from the simulator and recommends locker placement. The simulation models riders, battery physics in watt-hours, demand hotspots, locker inventory and charging, weather, and route constraints such as ferry crossings. A sensitivity analysis confirms the model responds correctly to its inputs. A gradient-boosted surrogate trained on 500 simulated scenarios predicts key outcomes accurately (mean stranded riders: $R^2 = 0.96$, Spearman 0.98), and a greedy optimiser using this surrogate produces locker placements that reduce stranded riders by $\sim 20\%$ versus random placement at an equal budget. We report the system architecture, validation, results, and a roadmap toward operator-data calibration.

Keywords: Agent-based simulation, Digital twin, Battery swapping, Facility location, Surrogate modelling, Gradient boosting, Urban mobility

CONTENTS

1	Introduction	1
2	Simulation Engine	1
2.1	Physical battery model	1
3	Calibration	1
4	Validation and Sensitivity Analysis	2
5	Experiment Framework and Data Generation	2
6	AI Decision-Support	2
6.1	Surrogate model	2
6.2	Objective and optimiser	3
7	Results	3
8	Discussion and Limitations	3
9	Future Work	3
10	Conclusion	4

1. INTRODUCTION

Urban e-bike delivery services depend on a reliable supply of charged batteries. Battery-swap operators provide networks of physical lockers where riders exchange a depleted battery for a charged one. For such an operator, the central planning questions are expensive and high-stakes: *where* should lockers be installed, *how many* are needed for a given level of demand, and *what capacity* each location requires. Testing these decisions in the field disrupts operations and is costly.

This project develops a *digital twin*: a simulation that reproduces the relevant dynamics of a swap network in a real city, plus an AI layer that learns from the simulation to recommend infrastructure decisions. This document reports the beta state of the system. Because the simulator is currently calibrated on documented best-estimates rather than operator data, results are presented as relative and structural; the architecture is designed so that real data can be substituted and the models retrained without redesign.

2. SIMULATION ENGINE

The simulator is agent-based [4] and organised in layers, each corresponding to a real-world planning concern.

- **City graph.** The Amsterdam bike network from OpenStreetMap via OSMnx [3], where nodes are intersections and edges are streets with real lengths; routing uses NetworkX [1]. The graph is restricted to its largest strongly connected component so every location is reachable.
- **Riders.** Agents that travel shortest (fastest) paths, consume battery in proportion to distance, seek a locker when low, and strand if depleted.
- **Demand.** Weighted hotspot zones (city centre, dining districts, stations) that drive where deliveries originate and end.
- **Lockers.** Finite-capacity stations (7, 8, or 10 slots) with time-based charging of returned batteries.
- **Conditions.** Weather (rain, wind, snow, heat) scales riding speed and battery drain; route constraints include the Centraal-Noord ferry.

2.1. Physical battery model

Battery state is tracked in watt-hours so values are checkable against real specifications. With capacity C (Wh) and consumption c (Wh/km), the full range is given by Equation 1.

$$R = \frac{C}{c} \quad (1)$$

The beta uses $C = 500$ Wh and $c = 12$ Wh/km, giving $R \approx 42$ km (about 33 km to the 20% reserve threshold). Edge travel time follows $t = \ell/v$ for edge length ℓ and speed v ; weather multiplies both travel time and battery cost.

3. CALIBRATION

All physical and behavioural parameters are documented best-estimates pending operator data (Table 1). They produce directionally-

^{*}Corresponding author.

This is a preliminary (beta) report. All simulation parameters are documented best-estimates pending real operator data; results are therefore to be read as *relative* and *structural* rather than absolute predictions.

correct behaviour but are not yet validated for absolute accuracy. The architecture isolates these values in configuration so that real measurements can replace them and the pipeline can be re-run.

Table 1. Key beta parameters (best estimates).

Parameter	Value
Battery capacity	500 Wh
Energy consumption	12 Wh/km
Reserve threshold	20%
Rider speed	18 km/h
Charge time (per battery)	3.5 h
Locker capacities	7 / 8 / 10 slots
Time step	10 s

Note: see the project calibration document for the basis and confidence level of each value.

4. VALIDATION AND SENSITIVITY ANALYSIS

Before any learning, a one-at-a-time sensitivity analysis confirms the simulator responds in the correct direction and magnitude. Fig. 1 ranks parameter impact on service failures. The key findings: adding lockers reduces stranded riders and raises swap-success rate; increasing rider count raises throughput but also strandings; worsening weather reduces throughput. Impact on strandings ranks as *demand* $\gg$ *locker count* $>$ *speed* $>$ *weather* — i.e. the two business levers (demand load and supply) are correctly dominant.

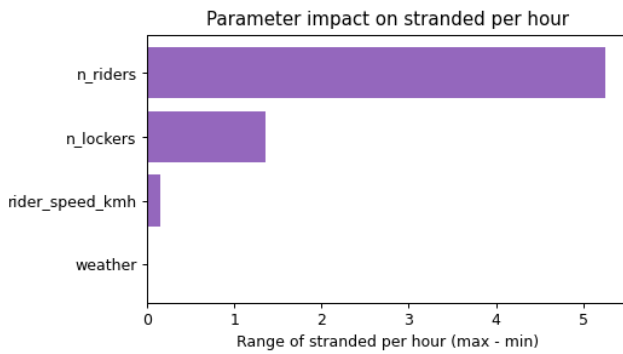


Figure 1. Parameter impact on stranded riders per hour (range of the mean across each parameter's swept values).

5. EXPERIMENT FRAMEWORK AND DATA GENERATION

A reproducible experiment framework turns the simulator into a training dataset. Candidate locker sites are generated from demand hotspots plus farthest-point spread. A sampler draws random combinations of *layout* (a subset of candidate sites) and *scenario* (rider count, weather, speed); a driver runs each to steady state and records one row of engineered features and outcome metrics. The graph is loaded once and runs execute in parallel across CPU cores. For this report, 500 scenarios were generated on the Amsterdam network.

Each row's *features* describe the layout's relationship to demand (coverage within travel-time thresholds, demand-weighted distance to the nearest locker, dispersion, supply ratios) plus the scenario conditions. The *targets* are normalised, steady-state outcome metrics (stranded riders, swap-success rate, throughput, locker utilisation, battery level).

6. AI DECISION-SUPPORT

The AI is two distinct components separated by the dataset: a *surrogate* that learns outcomes, and an *optimiser* that searches placements using the surrogate.

6.1. Surrogate model

A multi-output gradient-boosted regressor (one model per target, scikit-learn [2]) maps features to outcomes. Evaluation uses a group-wise train/test split (replicate seeds never cross the split). Fig. 2 and Fig. 3 show accuracy by target. Outcomes that vary smoothly with demand and supply are predicted very well (mean stranded riders $R^2 = 0.96$, battery 0.92, throughput 0.81); noisy failure-rate metrics are only moderately predictable ($R^2 \approx 0.5$).

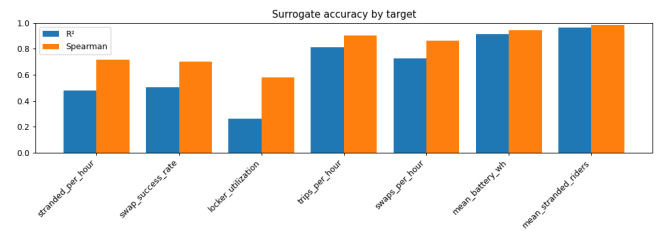


Figure 2. Surrogate accuracy (R^2 and Spearman rank correlation) per target on the held-out test set.

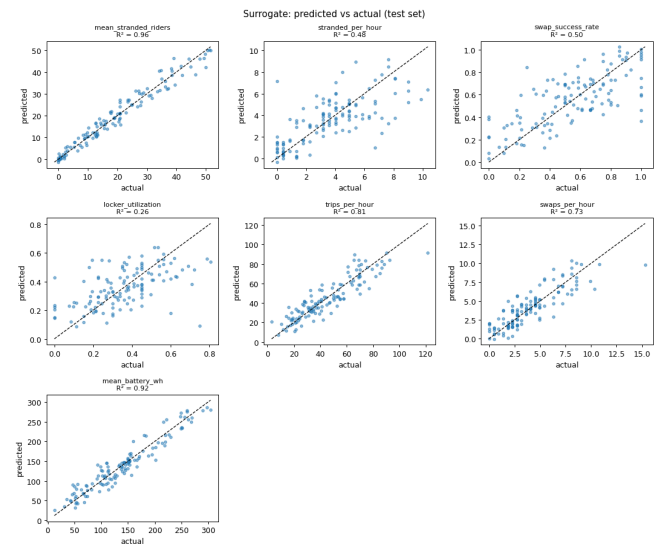


Figure 3. Predicted versus actual values on the test set; the dashed line is perfect prediction.

Permutation feature importance (Fig. 4) shows that stranded riders are driven primarily by rider count and the lockers-per-rider ratio, with spatial features contributing a smaller share — the model is strongest at network sizing.

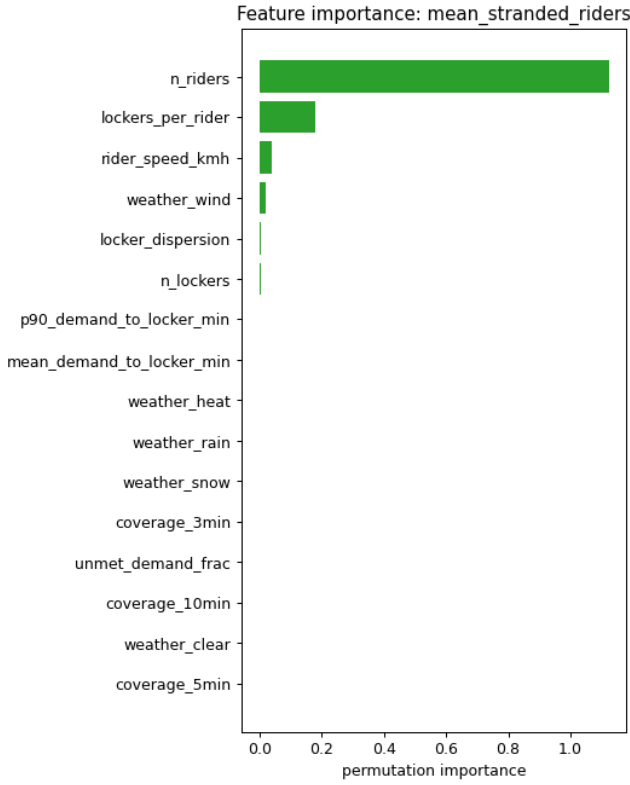


Figure 4. Permutation feature importance for mean stranded riders.

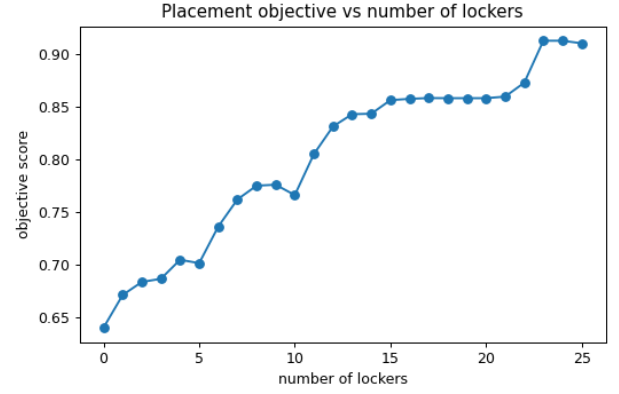


Figure 5. Objective score versus number of lockers (greedy placement).

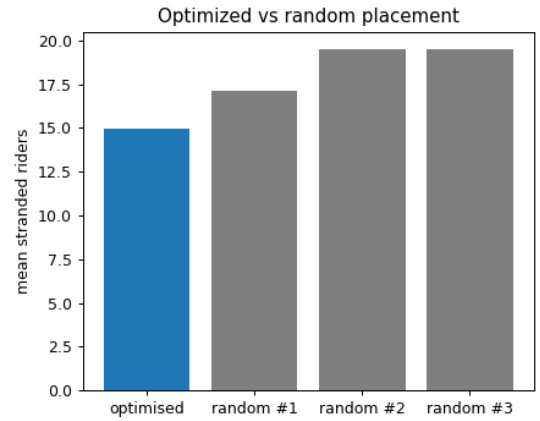


Figure 6. Optimised versus random placement at a six-locker budget (mean stranded riders, seed-averaged).

6.2. Objective and optimiser

Candidate placements are scored by a single objective combining the well-predicted, non-degenerate service signal (mean stranded riders, minimised) with optional secondary terms. Each target is normalised to $[0, 1]$ and minimisation targets are flipped, giving the weighted score in Equation 2, where g_i is the normalised, direction-adjusted value of predicted target $\hat{y}_i$ and w_i its weight.

$$S(\hat{y}) = \frac{1}{\sum_i w_i} \sum_i w_i g_i(\hat{y}_i) \quad (2)$$

A forward-greedy optimiser repeatedly adds the candidate site that most improves S , producing both a recommended layout and the full score-versus-budget curve.

7. RESULTS

Fig. 5 shows the objective rising with the number of lockers and plateauing — a clear diminishing-returns curve that answers “how many lockers”. To test placement *quality* at a fixed budget, we compared the optimiser’s six-locker recommendation against random six-locker layouts, averaged over three seeds to suppress run-to-run noise (Fig. 6). The optimised placement yields 14.96 mean stranded riders versus 18.72 for random — a ~20% reduction, and it beats every random layout tested. Table 2 summarises the headline metrics.

Table 2. Headline beta results.

Result	Value
Surrogate R^2 (mean stranded riders)	0.96
Surrogate Spearman (mean stranded)	0.98
Training scenarios	500
Placement improvement vs random	~20%
Automated tests passing	121

8. DISCUSSION AND LIMITATIONS

The pipeline works end-to-end and the closing-the-loop verification (running the recommended layout back through the simulator) confirms a genuine placement gain. Two qualifications are important. First, the system is calibrated on estimates, so results are relative, not absolute predictions. Second, the surrogate is strongest at *sizing* the network; the spatial signal that distinguishes good from bad *placement* is real but secondary, so the placement advantage is clearest at constrained budgets and must be measured with seed averaging. These are properties of the current data and metrics, not flaws in the architecture.

9. FUTURE WORK

Priorities for the next iteration: (i) obtain operator data (battery specs, fleet/demand figures, candidate sites, service-level targets) to calibrate and validate against real KPIs; (ii) strengthen the placement signal through more data, seed-replicated denoising, and richer spatial features; (iii) add locker capacity as a decision variable; and (iv) move toward a spatial (raster/CNN) representation enabling per-city “suitability heatmaps” for pre-deployment planning in new cities.

10. CONCLUSION

We have built and validated a digital-twin simulation of an e-bike battery-swap network and an AI pipeline that learns from it to recommend locker placement. On estimate-calibrated data the surrogate predicts the key service metric with high accuracy and the optimiser reduces stranded riders by roughly 20% versus random placement. The result is a credible beta and a clear path to an operator-calibrated decision-support tool.

■ REFERENCES

- [1] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using networkx”, in *Proceedings of the 7th Python in Science Conference*, 2008, pp. 11–15.
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, *et al.*, “Scikit-learn: Machine learning in Python”, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] G. Boeing, “Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks”, *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.
- [4] J. Kazil, D. Masad, and A. Crooks, “Mesa: An agent-based modeling framework”, in *Proceedings of the 19th Python in Science Conference*, 2020, pp. 53–60.